

CSE 4345: Big Data Analytics

Week 3, Part 2 — Seminal Research & Case Study

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Today's Agenda

Seminal Paper Covered

- 1 Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). *The Google File System*. ACM SOSP 2003. Bolton Landing, NY.

Case Study

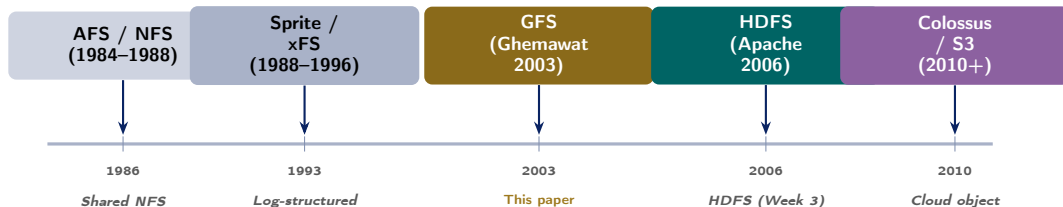
Yahoo! Hadoop Cluster (2006–2011) — the first industrial-scale open-source distributed filesystem deployment, scaling from 1,000 to 42,000 nodes in five years.

CLO Addressed

CLO1 — Understand distributed storage architecture

Research Paper — Ghemawat, Gobioff & Leung (2003)

Research Landscape: Distributed File Systems Timeline



Intellectual Lineage

AFS showed network-transparent file access → xFS showed log-structured, distributed metadata → GFS (2003) discarded POSIX to fit Google's actual workload → HDFS replicated the GFS design in Java for open-source → Cloud object storage (S3, Azure Blob) extended the large-block philosophy globally.

Ghemawat et al. (2003): Publication Context

Full Citation

Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). **The Google File System**. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pp. 29–43. Bolton Landing, NY, USA.

Venue: ACM SOSP — Rank A* (CORE), premier OS venue

Cited by: >14,000 (Google Scholar, 2024)

Authors: Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung (Google Infrastructure Team)

Historical Moment

Google had already been running GFS internally since 2000. Publishing in 2003 opened the design to the world. Doug Cutting read the paper in 2004 and built

The Core Design Philosophy

“Component failures are the norm, not the exception.”

Google ran GFS on **thousands of commodity Linux PCs**. Mean time between failures for a single machine was measured in *weeks*, not years. The filesystem had to tolerate failures *transparently* — without operator intervention.

The Disruptive Choice

GFS deliberately **violated POSIX semantics** (no random writes, relaxed consistency) to achieve performance and fault tolerance at scale. Prior distributed filesystems tried to emulate POSIX exactly — and failed at scale as a result.

Technical Contribution 1 — GFS Architecture

Three-Component Design

- **Single Master**

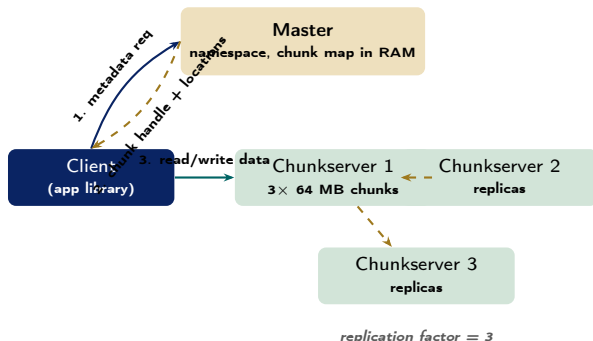
Stores all metadata in RAM: file namespace, chunk locations, chunk-to-server mapping. Never on the data read/write path.

- **Chunkservers** (thousands)

Each stores 64 MB chunks as plain Linux files. No awareness of file semantics — only chunks and checksums.

- **Clients**

Ask master for chunk handle + chunkserver addresses, then communicate *directly* with chunkservers. Master is **never a data bottleneck**.



Technical Contribution 2 — Fault Tolerance & Consistency

Replication and Recovery

- Every chunk replicated to **3 chunkservers** by default
- **Rack-aware placement:** primary replica on Rack A, two secondaries on distinct Rack B / C to survive rack-level power or switch failure
- Master heartbeats every chunkserver every **30 seconds**; missing 3 heartbeats → re-replication begins automatically
- **Checksumming:** each 64 KB block within a chunk carries a 32-bit CRC; verified on every read — silent corruption caught
- **Stale replica detection:** chunk version numbers incremented on every lease grant; stale replicas are garbage collected

GFS Consistency Model

Operation	Consistent?	Defined?
Serial write	Yes	Yes
Concurrent write	Yes	No
Serial record append	Yes	Yes
Concurrent rec. append	No	At records

Atomic Record Append

Multiple clients append to the *same* file concurrently. GFS guarantees each append is written **atomically at least once** at an offset chosen by the primary chunkserver. Readers may see padding or duplicates — handled by

Technical Contribution 3 — Master Design & Leases

Single-Master Metadata Management

- All file namespace and chunk-location metadata kept **in master RAM**: $<64 \text{ bytes per chunk} \times \text{millions of chunks} = \text{a few GB}$, affordable in 2003
- Namespace and chunk-ownership changes are logged to a **write-ahead operation log** on disk and replicated to **shadow master** replicas
- On master restart, the log is replayed; chunk locations are **not persisted** — chunkservers report their chunk inventory at startup (simpler, always fresh)
- Concern: single point of failure?
Master restarts in **seconds** from log; shadow masters serve *read-only* metadata during this window

Leases: Coordinating Concurrent Mutations

- ❶ Client requests write → asks master for chunk lease
- ❷ Master grants **60-second lease** to one “primary” chunkserver
- ❸ Primary defines a **serial mutation order**; secondaries follow that order
- ❹ Data flows in a **pipeline**: client → nearest chunkserver → next → next (chain replication)
- ❺ Lease can be **extended** while writes are active; expires if master does not renew

GFS vs. HDFS — Design Differences

Design Dimension	GFS (2003)	HDFS (2006+)
Language	C++	Java (JVM)
Default chunk size	64 MB	128 MB (configurable)
Metadata store	Master RAM + log	NameNode RAM + edit log + FSImage
HA for master	Shadow masters	Active/Standby NameNode (HDFS 2)
Snapshots	Supported (COW)	Supported
Random writes	No (append-only)	No
Checksumming	Per 64 KB block	Per 512-byte chunk
Replication	3 (rack-aware)	3 (rack-aware)
Erasur coding	No (GFS II / Colossus)	Yes (HDFS 3 EC)
Open source	No (proprietary)	Yes (Apache License 2.0)

Key Lesson

Direct Descendants

- **HDFS** (2006) — the most deployed distributed filesystem in enterprise Big Data
- **Colossus** — Google's internal GFS successor (2010+); adds erasure coding, faster master
- **Amazon S3** (2006) — large-object storage shares the 64 MB+ block philosophy; immutable objects match GFS's append-only model
- **Azure Blob Storage / Google Cloud Storage** — same architectural lineage
- **Delta Lake / Apache Iceberg** (2019+) — write immutable Parquet files to object storage; the “no random writes” constraint is now a *feature*

Stanford CS246 / MIT 6.830 Perspective

- Stanford CS246 uses GFS as the canonical example of **system design under failure assumptions**: fault-tolerance via replication, not RAID
- MIT 6.830 covers the GFS consistency model as an instance of **relaxed consistency for throughput**: trading “defined” for “consistent” under concurrent appends
- CMU 15-721 frames GFS chunk servers as the earliest example of **shared-nothing distributed storage** for analytical workloads

Case Study — Yahoo! Hadoop: 42,000 Nodes in Five Years

The Yahoo! Data Problem (2005)

Why Yahoo! Needed a New Storage Layer

In 2005 Yahoo! operated:

- **Web crawler** (Nutch) producing 20+ TB/day of crawl data — unmanageable on a single-server NFS mount
- **Ad click logs**: 15 billion events per day requiring daily aggregation for billing
- **Email spam filter**: real-time classification over a corpus of 200M accounts
- **Personalisation**: user-activity logs for Yahoo! News, Finance, Sports — terabytes per property
- All of these ran on *separate, siloed* infrastructure; sharing compute capacity was impossible

The Turning Point

In January 2006, Yahoo! hired **Doug Cutting** — the creator of Apache Lucene and Nutch, who had already built an early HDFS prototype (then called NDFS) while reading the GFS and MapReduce papers.

Yahoo! gave Cutting a team and a mandate: *build an open-source distributed computing platform that can run on Yahoo!'s existing commodity Linux clusters.*

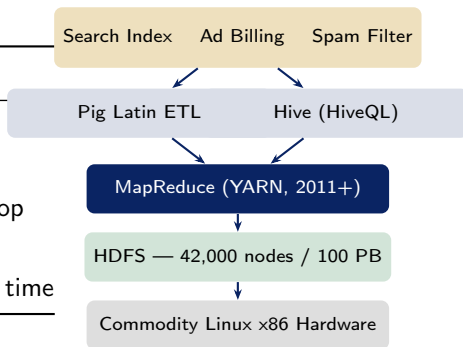
Project Name Origin

Doug Cutting named the project **Hadoop** after his son's toy yellow elephant — a name chosen because it was short, pronounceable,

Yahoo! Hadoop Architecture (2008–2011)

Cluster Growth Timeline

Year	Nodes	Milestone
2007	1,000	First production cluster
2008	4,000	1 TB sort in 209 s (world record)
2009	10,000	Search index generation moved to Hadoop
2010	26,000	100 PB stored; daily job count >10,000
2011	42,000	Largest Hadoop cluster ever built at the time



Hardware Specification (2008 commodity node)

- 2× quad-core Intel Xeon @ 2.0 GHz
- 8 GB RAM

Yahoo!'s Open-Source Contributions from Hadoop Operations

What Yahoo! Built and Donated to Apache

Operating 42,000 nodes revealed gaps that Yahoo! engineers filled and then open-sourced:

- **Apache Pig** (2008) — dataflow language (Pig Latin) for ETL pipelines without Java MapReduce
- **Apache ZooKeeper** (2008) — distributed coordination service (named after the Hadoop zoo of projects); now used by Kafka, HBase, Pulsar
- **Apache Oozie** (2010) — workflow scheduler for DAGs of Hadoop jobs (replaced cron for multi-stage pipelines)
- **YARN** (2012, Hadoop 2.0) — resource manager that decoupled cluster resource management

Why Open Source Mattered

Yahoo! made a strategic bet: by open-sourcing Hadoop, they would **attract external developers** to fix bugs and add features without Yahoo! paying for the work.

This bet paid off:

- 1 Facebook, LinkedIn, Twitter adopted Hadoop — and contributed back Hive, Kafka, and Storm respectively
- 2 Cloudera (2008) and Hortonworks (2011) formed as commercial Hadoop distributors
- 3 1,000+ contributors joined the Apache Hadoop project by 2013

Lessons Learned & Discussion Questions

Key Lessons from the Yahoo! Case

- 1 **Commodity hardware wins at scale:** 42,000 low-cost nodes outperformed any single enterprise server at equivalent cost; failures became routine and manageable
- 2 **GFS design scales to the real world:** HDFS implemented GFS faithfully and handled 100 PB without fundamental redesign
- 3 **Open source creates ecosystems:** Yahoo!'s Pig, ZooKeeper, and Oozie contributions seeded the entire modern data engineering toolchain
- 4 **Workload shapes the API:** MapReduce's rigid two-phase model drove the need for Pig (higher-level) and later Spark (in-memory, multi-step DAGs)

Discussion Questions (CLO1, CLO4)

- 1 GFS uses a **single master** holding all metadata in RAM. Yahoo! eventually ran HDFS with a single NameNode on a 42,000-node cluster. Identify **two scalability bottlenecks** this creates and explain how HDFS 2 (YARN era) addressed them. *[CLO1 — Understand]*
- 2 Yahoo!'s 2008 TeraSort sorted 1 TB in 209 s on 910 nodes, achieving ≈ 5 GB/s total throughput. If each node has a 1 Gbps NIC, calculate the **theoretical maximum network throughput** and explain why actual throughput was lower. *[CLO4 — Analyse]*
- 3 Yahoo! chose to **open-source Hadoop**

Live Exercise

Live Exercise — HDFS Block Simulation in Python

Task (25 minutes, pairs)

Simulate the **GFS/HDFS block placement algorithm** without a real cluster.

Goal: given a 500 MB file and 64 MB blocks, distribute 3 replicas rack-aware across 6 nodes (2 racks, 3 nodes each). Print the block map.

Rack-aware rule:

- 1 Replica 1: random node on Rack A
- 2 Replica 2: random *different* node on Rack A
- 3 Replica 3: random node on Rack B

Extension: simulate a node failure and trigger re-replication to maintain factor = 3.

Python Skeleton

```
import random, math

BLOCK_SIZE = 64 # MB
FILE_SIZE = 500 # MB
REPL = 3

racks = {
    "A": ["nodeA1", "nodeA2", "nodeA3"],
    "B": ["nodeB1", "nodeB2", "nodeB3"],
}

n_blocks = math.ceil(FILE_SIZE / BLOCK_SIZE)
block_map = {}

for b in range(n_blocks):
    r1 = random.choice(racks["A"])
    r2 = random.choice(
        [n for n in racks["A"] if n!=r1])
    r3 = random.choice(racks["B"])
    block_map[f"block_{b}"] = [r1, r2, r3]

for blk, nodes in block_map.items():
    print(f"{blk}: {nodes}")
```

Seminal Paper

- Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google File System. *Proceedings of the 19th ACM SOSP*, 29–43.

Textbooks [TW], [SA], [TE]

- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 3 — HDFS; Ch. 2 — MapReduce Overview). O'Reilly.
- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 3 — Hadoop Ecosystem). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 5 — Distributed Storage). Prentice Hall.

Case Study Sources

- Owen, S., Anil, R., Dunning, T., & Friedman, E. (2011). *Mahout in Action* (Yahoo! Hadoop case). Manning Publications.

Key Takeaways

- GFS (2003) proved that **commodity hardware + software fault-tolerance** outperforms expensive reliable hardware at web scale — the foundational insight of distributed storage
- **HDFS is GFS** re-implemented in Java; understanding GFS means understanding HDFS at depth
- Yahoo!'s 42,000-node cluster validated GFS design at industrial scale *and* produced most of the tools (Pig, ZooKeeper, YARN, Oozie) we still use today
- The **append-only, large-block** design choice — made in 2003 for MapReduce — is still the dominant pattern in cloud object storage and Delta Lake today

Quote of the Week

*“The system is designed for large streaming reads, not small random reads.
Sustained bandwidth matters more than low latency.”*

— Ghemawat, Gobioff & Leung (2003)

Part 1 Preview — Week 4

MapReduce, Pig, Hive & HBase

- MapReduce execution model and fault tolerance
- Pig Latin for ETL pipelines